SLOVENSKÁ TECHNICKÁ
UNIVERZITA V BRATISLAVE
FAKULTA INFORMATIKY
A INFORMAČNÝCH TECHNOLÓGIÍ

MNIST Klasifikátor (3.a) + Backpropagácia

Meno: Juraj Černička

ID: 127128

Rok: 2024/25

Cvičenie: Ing. Martin Komák, PhD., Štvrtok 16:00

MNIST klasifikátor

Problém

Cieľ zadania je navrhnúť neurónovú sieť, ktorej implementáciu vytvoríme pomocou knižnice PyTorch. Túto neurónovú sieť treba následne natrénovať na obrázkoch ručne písaných číslic s rozmermi 28 * 28 pixelov.

Tieto obrázky aj so správnymi výsledkami pre každý obrázok zoberieme z dátového súboru MNIST. Dáta sú rozdelené na dve množiny: trénovacia množina obsahuje 60 000 obrázkov, zatiaľ čo testovacia obsahuje 10 000 obrázkov.

Túto neurónovú sieť potreba vyskúšať s tromi optimalizer algoritmami: SGD, SGD s momentom a ADAM. Výsledky jednotlivých algoritmov ako aj výsledky s rôznymi hyperparametrami porovnáme. Môžeme porovnať aj rôzne architektúry neurónovej siete.

Opis riešenia

Dáta transformujeme pomocou transformačných funkcií ToTensor() a Normalize(). Tieto zmeny na vstupných dátach vykonáme, aby sme dostali vstupy, na ktorých je neurónovú sieť jednoduchšie trénovať.

ToTensor() zmení 256 možných hodnôt pixelov na hodnoty medzi 0 a 1. Normalize() zmení hodnoty takým spôsobom, aby sa nachádzali v intervale od -1 po 1 s tým, že priemerná hodnota je 0.

Neurónovú sieť reprezentujeme nami deklarovanou podtriedou triedy nn.Module. Táto podtrieda v konštruktore vytvára parametre jednotlivých neurónových vrstiev.

Tiež vytvárame definíciu funkcie forward, ktorá vstupné hodnoty najprv sploští, aby ich mohla následne poslať ako vstupy do neurónovej siete.

Následne zvlášť na všetkých troch optimalizačných algoritmoch otestujeme našu neurónovú sieť. Najprv cez ňu pošleme vstupy trénovacej množiny. Pri posielaní týchto vstupov rozdelíme dáta vstupov do skupiniek: batchov. Po otestovaní funkčnosti pre každý prvok v batchi sa na základe výsledkov vyhodnotenia tohto batchu vykoná spätný chod, v ktorom sa aktualizujú hodnoty neurónovej siete.

Týmto spôsobom sa vďaka opakovaným aktualizáciam parametrov neurónová sieť postupne učí podľa trénovacej množiny predpovedať správnu hodnotu. Po vyplytvaní celej trénovacej množiny sa úspešnosť neurónovej siete otestuje na testovacej množine.

Pri testovaní sa neurónová sieť zo svojich chýb nepoučuje. Po vyplytvaní trénovacej aj testovacej množiny ubehla jedna epocha, a celý proces sa zopakuje.

Reprezentácia údajov

Na reprezentáciu obrázku ručne napísanej číslice sa používajú transformované hodnoty pixelov. Tieto hodnoty sú v intervale -1 až 1, s priemernou hodnotou rovnej nule. Matica transformovaných hodnôt treba sploštiť, aby mohla byť správne spracovaná cez neurónovú sieť. Hodnoty budú teda uložené vo vektore, kde každá hodnota sa mapuje na hodnotu unikátneho pixela.

Pre reprezentáciu modelu používame triedu MNISTClassifier, ktorá obsahuje jednotlivé vrstvy siete. Tieto vrstvy sa priamo nemapujú na vrstvy v neurónovej sieti. Namiesto toho každá vrstva reprezentuje prechod medzi dvoma neurónovými vrstvami.

Navyše každý prechod sa môže skladať z viacerých vrstiev. Lineárna vrstva aplikuje výpočet hodnôt nasledovných neurónov pomocou matice váh a matice biasov. O vnútornú architektúru sa stará knižnica PyTorch.

Experimenty

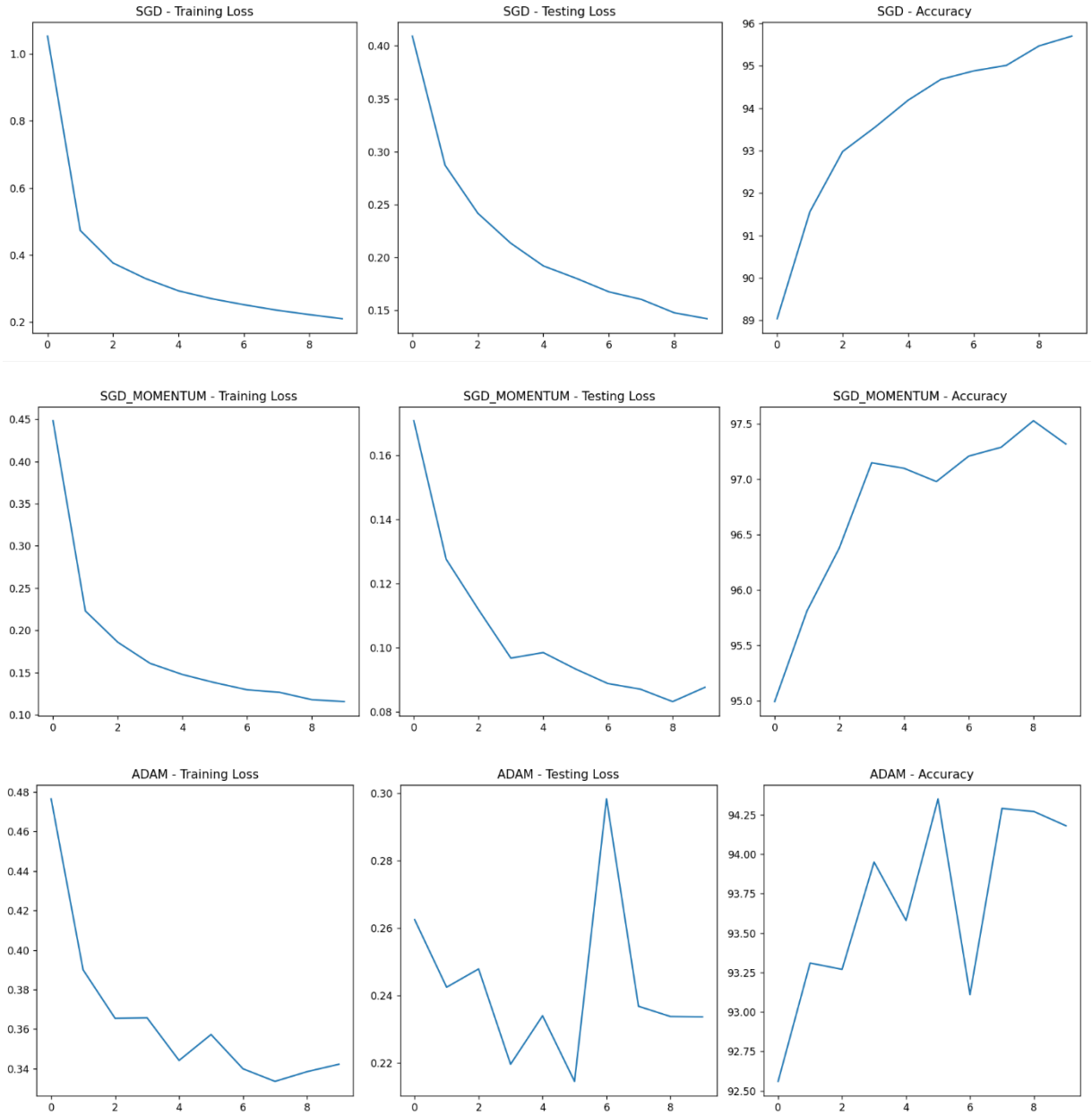
Niektoré parametre boli medzi jednotlivými experimentami nemenené:

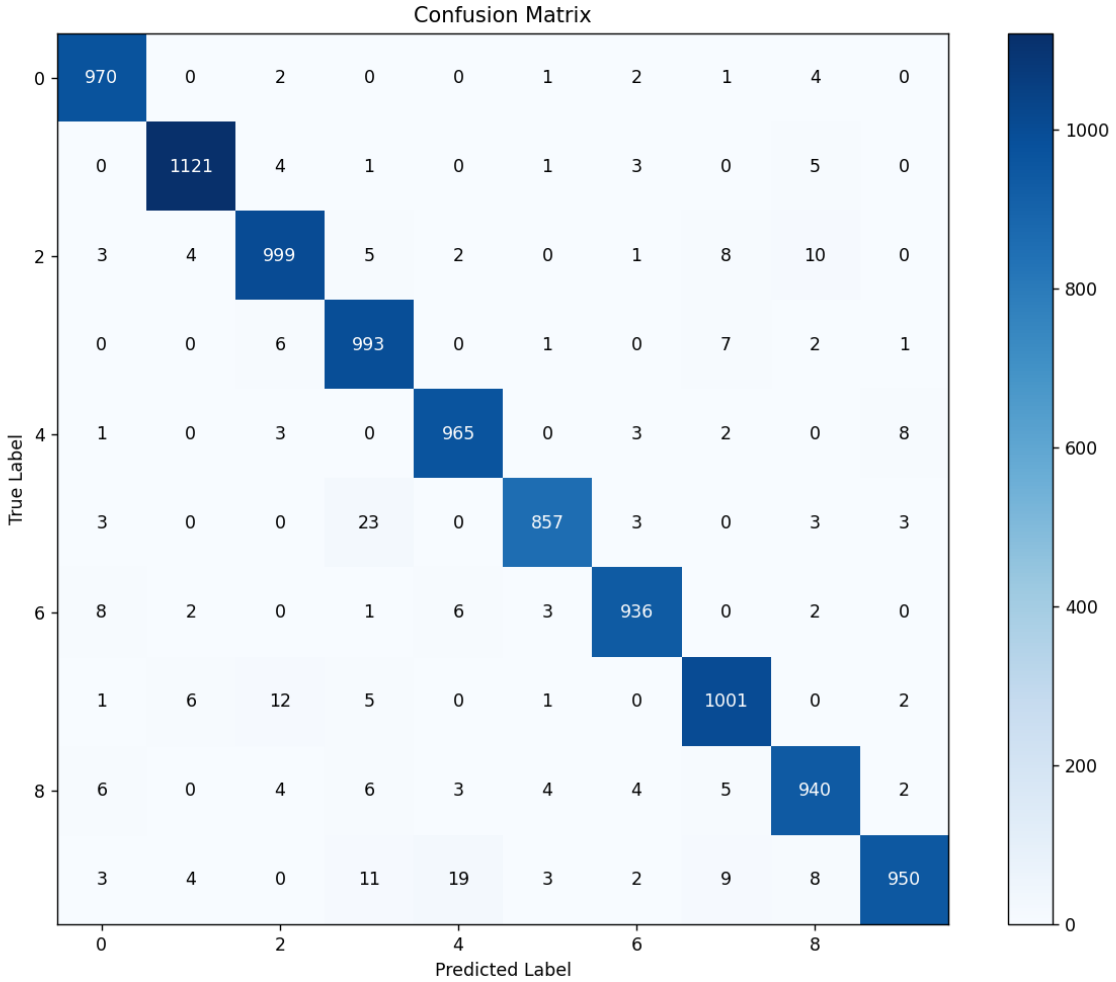
Loss funkcia	Veľkosť batchu
Cross entropy loss	64

Výsledky experimentov vykonané pre rôzne kombinácie parametrov:

Experiment 1:

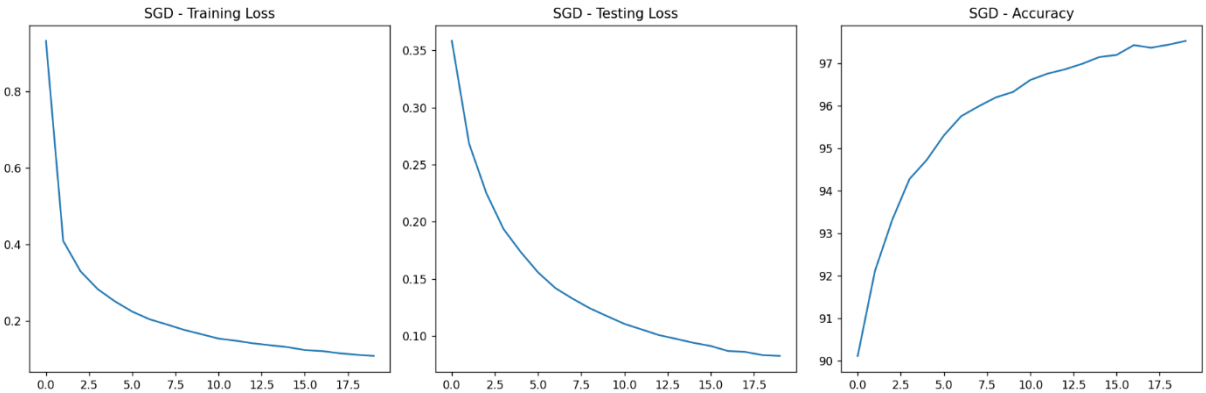
Veľkosti skrytých vrstiev	Aktivačné funkcie	Dropout	Počet epoch	Learning rate	Momentum
60, 60	ReLU	0.2	10	0.01	0.9

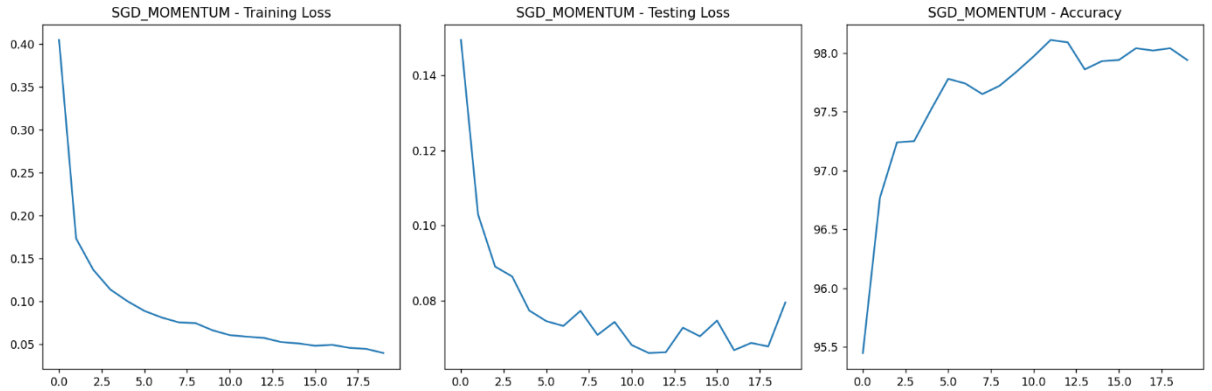

SGD - Training Loss
SGD - Testing Loss
SGD - Accuracy
SGD_MOMENTUM - Training Loss
SGD_MOMENTUM - Testing Loss
SGD_MOMENTUM - Accuracy
ADAM - Training Loss
ADAM - Testing Loss
ADAM - Accuracy

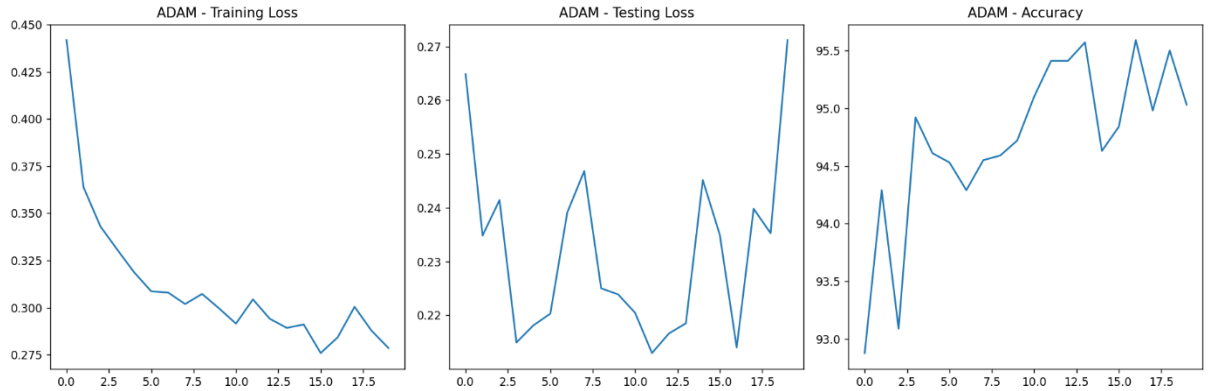


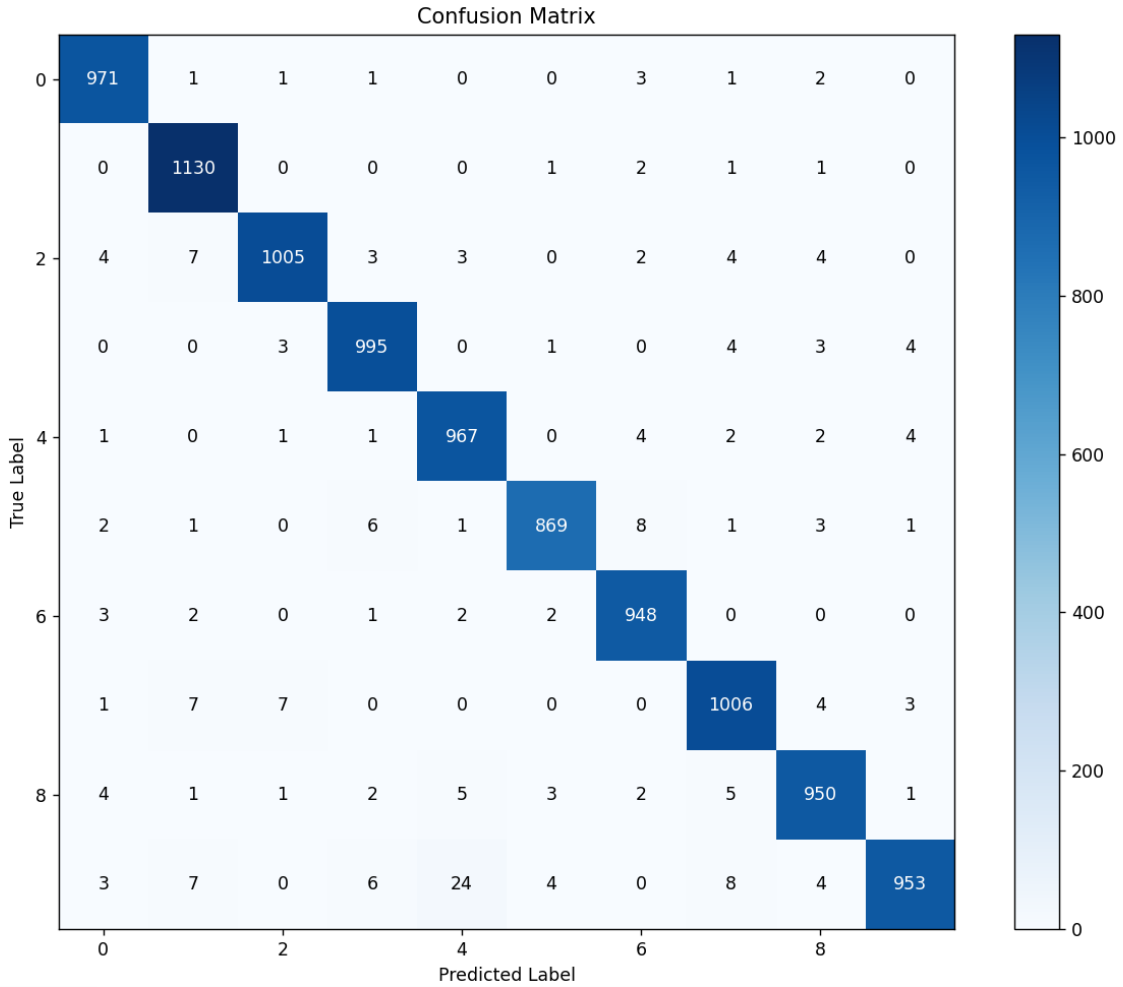
Experiment 2:

Veľkosti skrytých vrstiev	Aktivačné funkcie	Dropout	Počet epoch	Learning rate	Momentum
128, 64	ReLU	0.2	20	0.01	0.9



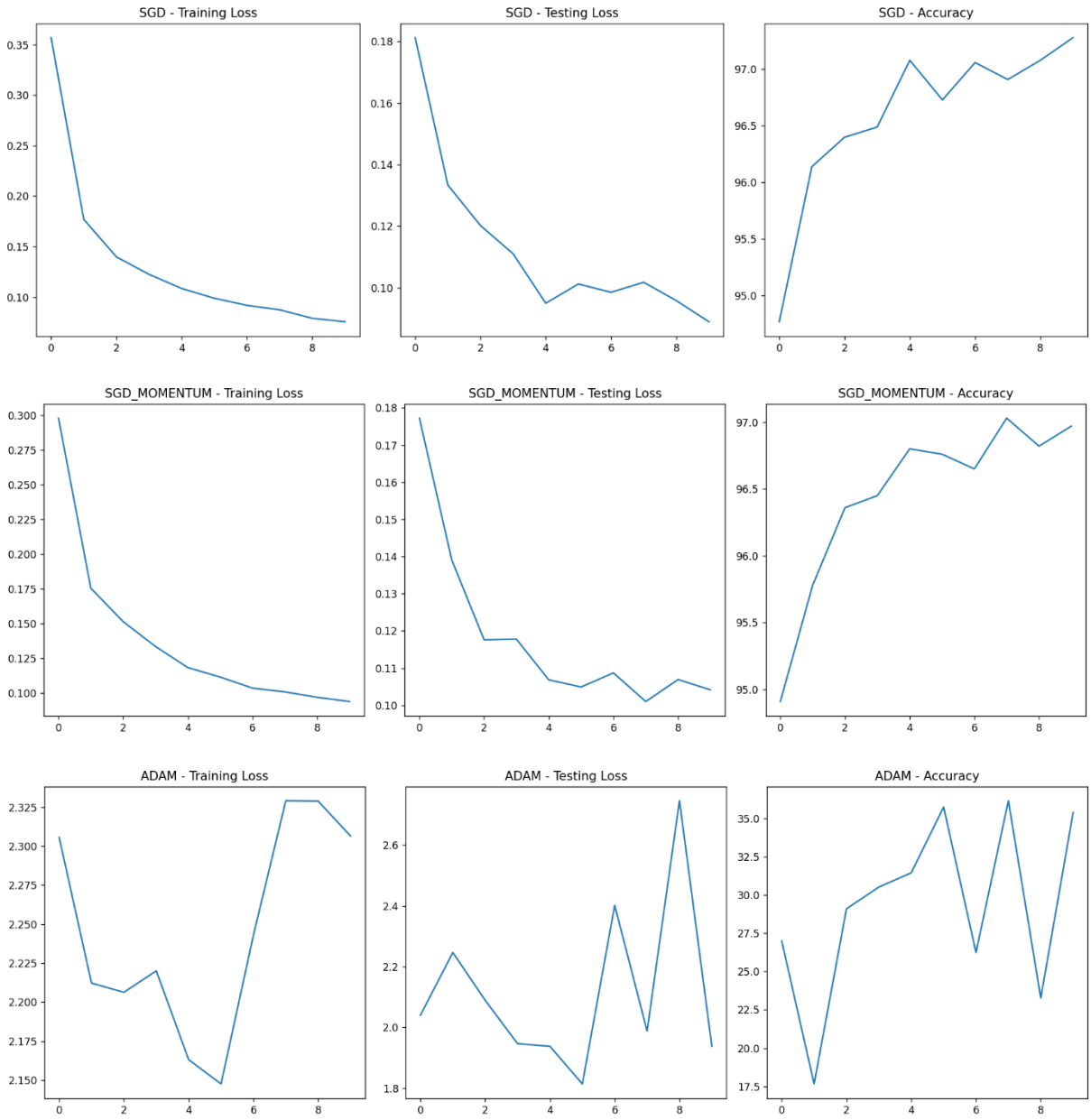

SGD_MOMENTUM - Training Loss
SGD_MOMENTUM - Testing Loss
SGD_MOMENTUM - Accuracy

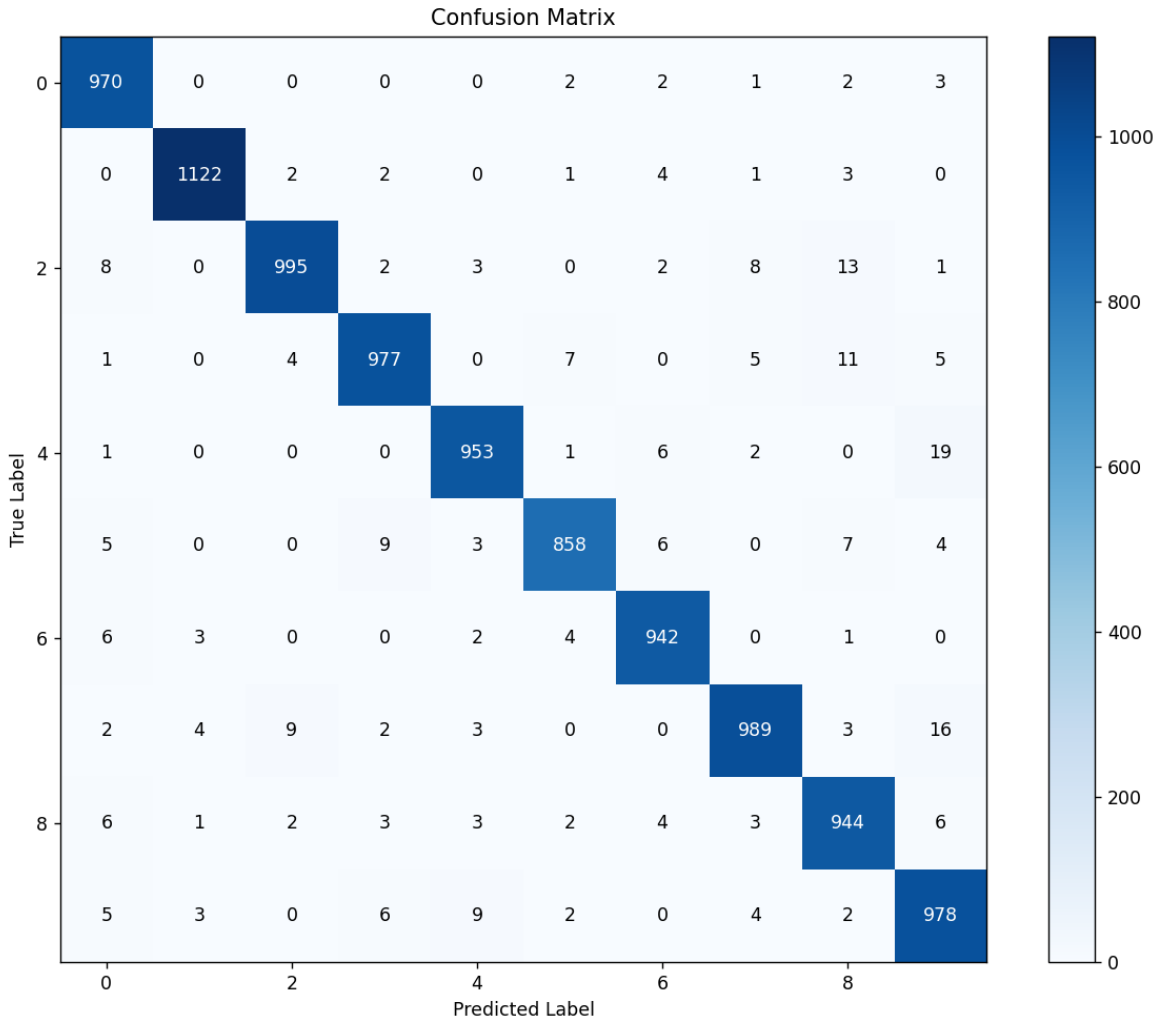

ADAM - Training Loss
ADAM - Testing Loss
ADAM - Accuracy


Confusion Matrix
True Label
Predicted Label
971 1 1 1 0 0 3 1 2 0
0 1130 0 0 0 1 2 1 1 0
4 7 1005 3 3 0 2 4 4 0
0 0 3 995 0 1 0 4 3 4
1 0 1 1 967 0 4 2 2 4
2 1 0 6 1 869 8 1 3 1
3 2 0 1 2 2 948 0 0 0
1 7 7 0 0 0 0 1006 4 3
4 1 1 2 5 3 2 5 950 1
3 7 0 6 24 4 0 8 4 953

Experiment 3:

Veľkosti skrytých vrstiev	Aktivačné funkcie	Dropout	Počet epoch	Learning rate	Momentum
60, 60	Tanh	0.1	10	0.1	0.8





Porovnanie experimentov

Pri experimentovaní s hodnotami parametrov neurónovej siete sme zistili, že algoritmus ADAM funguje vo všetkých otestovaných prípadoch najhoršie. Tento algoritmus je zároveň najcitlivejší na zmenu týchto parametrov, a preto je pri ňom kľúčové správne nastavenie počiatočných hodnôt.

SGD s momentom sa konzistentne umiestnil na prvom mieste, a najlepšie výsledky získal pri zväčšení learning rate. ADAM naopak dopadol po zväčšení learning rate katastrofálne, pretože je optimalizovaný na menšie hodnoty learning rate ako SGD a SGD s momentom.

Používateľská príručka

Po spustení programu sa v konzole po krátkej pauze pre načítanie dát vypíšu hodnoty pre prvú epochu. Následne sa začnú vypisovať hodnoty pre jednotlivé epochy hneď po ich ubehnutí.

Poradové číslo epochy je napísané na začiatku výpisu. Po vykonaní všetkých epoch sa nakreslí graf pre daný optimalizačný algoritmus. Po vypnutí grafu sa proces zopakuje pre nasledujúci algoritmus.

Po vykonaní všetkých troch algoritmov sa vypíše algoritmus, ktorý dosiahol najlepšie výsledky, a následne vygeneruje confusion matrix pre tento algoritmus.

Backpropagation

Problém

Cieľom zadania je vytvoriť vlastný algoritmus backpropagation, ktorý otestujeme na vlastnej implementácií neurónovej siete. Táto neurónová sieť bude obsahovať jednu skrytú vrstvu so štyrmi neurónmi.

Vstupná vrstva bude obsahovať dva neuróny a výstupná jeden neurón. Túto sieť natrénujeme na problémoch XOR, AND a OR. Pri trénovaní využijeme našu implementáciu backpropagácie.

Nemôžeme využívať knižnicu PyTorch ani iné podobné knižnice na neurónové siete.

Opis riešenia

Používame pole vstupných hodnôt pre testovanie a trénovanie modelu. Toto pole obsahuje štyri hodnoty, ktorými sú všetky možné kombinácie jednotiek a núl pre dva vstupy. Pole správnych výsledkov pre vstupy vyberieme podľa trénovanej operácie. Trénovanou operáciou môže byť XOR, OR alebo AND.

Ako chybovú funkciu používame MSE. Pre aplikáciu gradientov na zmenu jednotlivých váh používame algoritmus SGD s momentom. V prípade hodnoty 0 pre moment je použitý algoritmus obyčajný SGD.

Pre vykonanie predného chodu sa na vstupy postupne aplikujú transformácie váh jednotlivých prechodov medzi vrstvami. Tieto transformácie menia hodnoty vstupov podľa váh

a biasov v prechodoch. Taktiež menia rozmery vstupov a preto sa počet výstupov nemusí rovnať počtu vstupov.

Tieto transformácie sa aplikujú pomocou hodnôt v maticiach, ktoré predstavujú váhy jednotlivých prechodov. Biasy sú uložené vo vlastnej matici, ktorá sa k výsledkom pripočíta.

Na výsledok maticových operácií je aplikovaná aktivačná funkcia v danom prechode. Tá zaistí, že hodnoty neurónov v skrytých vrstvách budú v špecifických intervaloch.

Pri výpočte samotnej backpropagácie získavame z derivácií chybovej funkcie hodnoty gradientov váh a biasov. Tieto hodnoty po dokončení spätného chodu aplikujeme na hodnoty váh a biasov, aby sme zmenšili výsledok chybovej funkcie. Týmto systémom sa neurónová sieť postupne učí na trénovacích vzorkách.

Pokaždé, keď do neurónovej siete ako vstup vložíme všetky štyri vstupné hodnoty a vypočítame priemernú hodnotu chybovej funkcie, tak túto priemernú hodnotu použijeme na vypočítanie spätného chodu a následnej aktualizácie parametrov modelu. Vykonanie týchto akcií sa nazýva jeden epoch.

Reprezentácia údajov

Neurónová sieť je reprezentovaná triedou Model. Táto trieda obsahuje jednotlivé vrstvy siete, reprezentované pomocou inštancií tried rôznych tried, kde každá inštancia predstavuje časť prechodu medzi dvoma neurónovými vrstvami. Týmito triedami môže byť buď lineárna alebo aktivačná vrstva.

Lineárna vrstva predstavuje časť prechodu počítanú pomocou váh a biasov.

Aktivačná vrstva predstavuje časť prechodu aplikujúcu aktivačnú vrstvu. Táto aktivačná funkcia môže byť funkcia ReLU, Tanh alebo Sigmoid. Druh aktivačnej funkcie, ktorú aktivačná vrstva aplikuje, sa vyberá pri inicializácii inštancie tejto aktivačnej vrstvy.

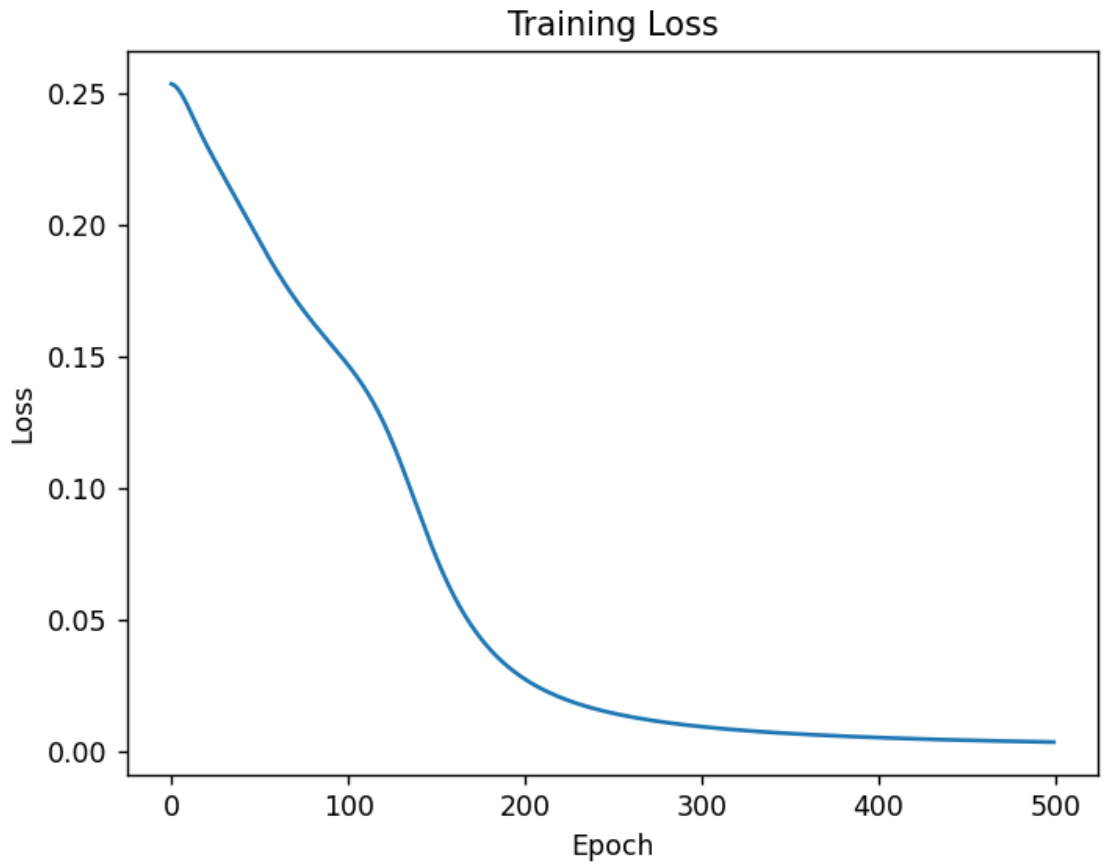
Váhy a biasy pre prechody medzi vrstvami sú uložené v maticiach. Gradienty pre váhy aj biasy sú taktiež uložené v maticiach. Tieto matice sa nachádzajú vnútri lineárnej vrstvy. Táto vrstva obsahuje aj predchádzajúce hodnoty gradientov.

Experimenty

Experimenty v prípade, že nie je uvedené inak, fungovali pre testovacie výstupy správne.

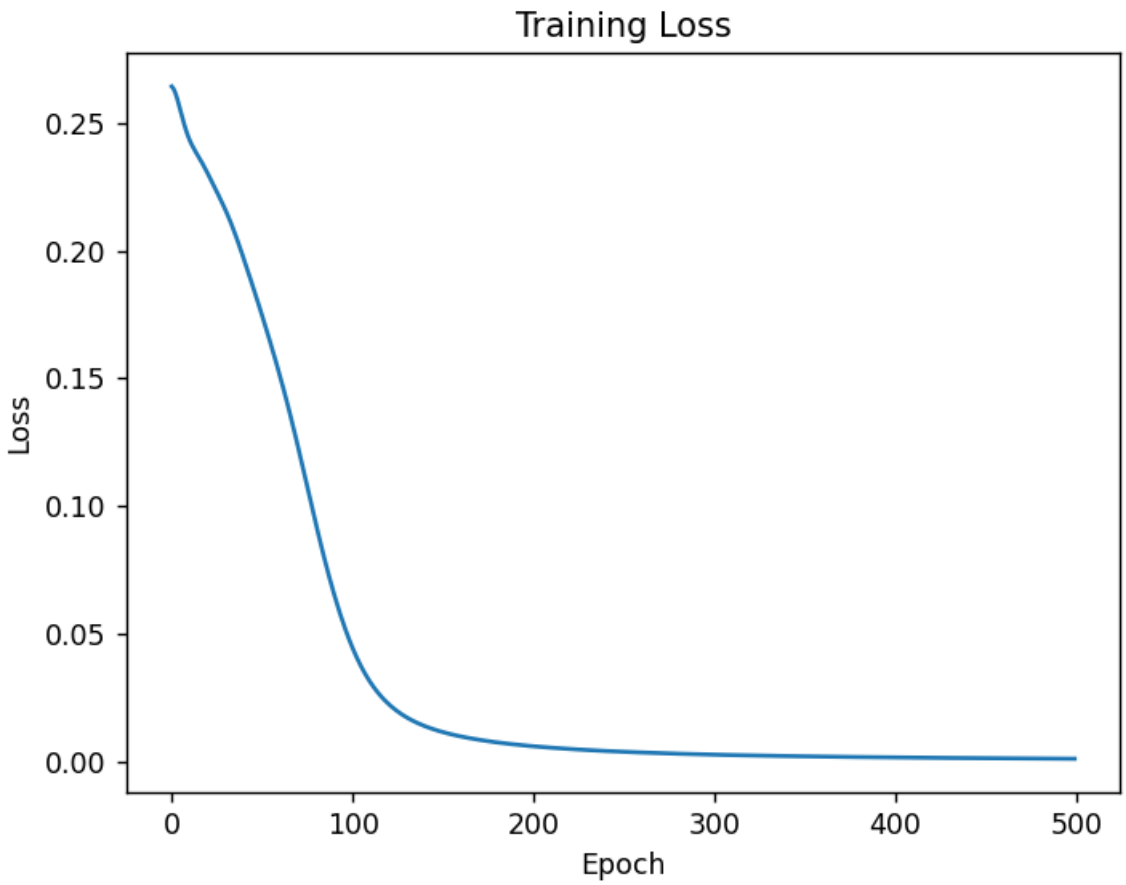
Experiment 1:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
XOR	0.9	4	0.8	Tanh



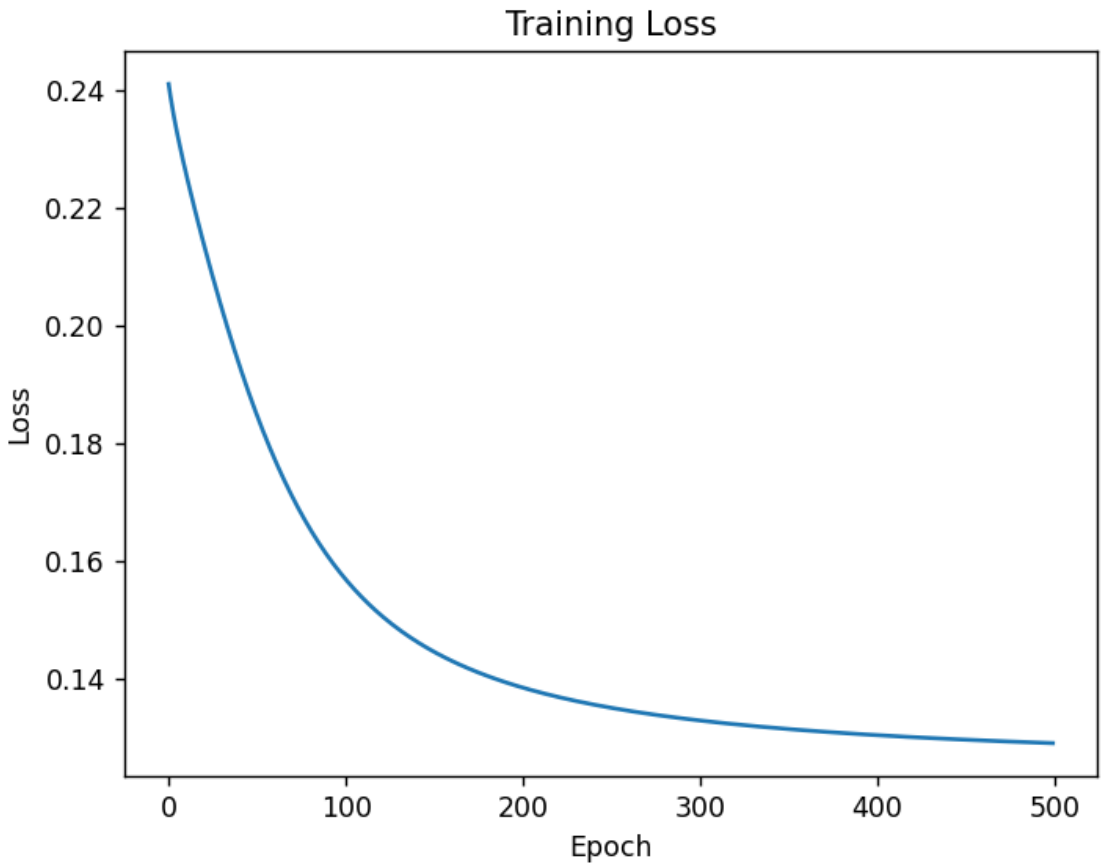
Experiment 2:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
XOR	0.9	4, 4	0.8	Tanh



Experiment 3:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
XOR	0	4	0.8	Tanh

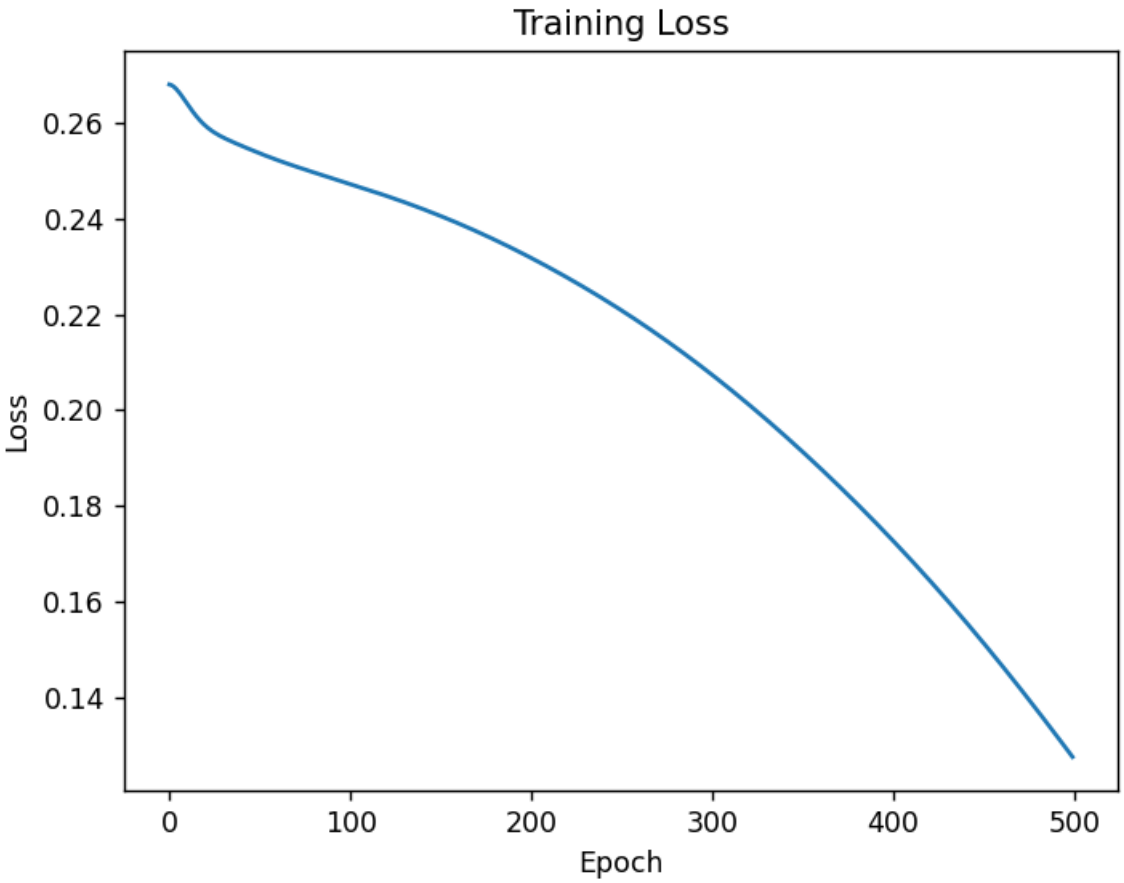



```
Predictions for a trained network:
[[0.04803232]
 [0.94060837]
 [0.49690699]
 [0.5068081 ]]
```

Pri trénovaní sme dostali nesprávne hodnoty (perfektné výsledky by boli 0, 1, 1, 0)

Experiment 4:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
XOR	0.9	4	0.2	Tanh

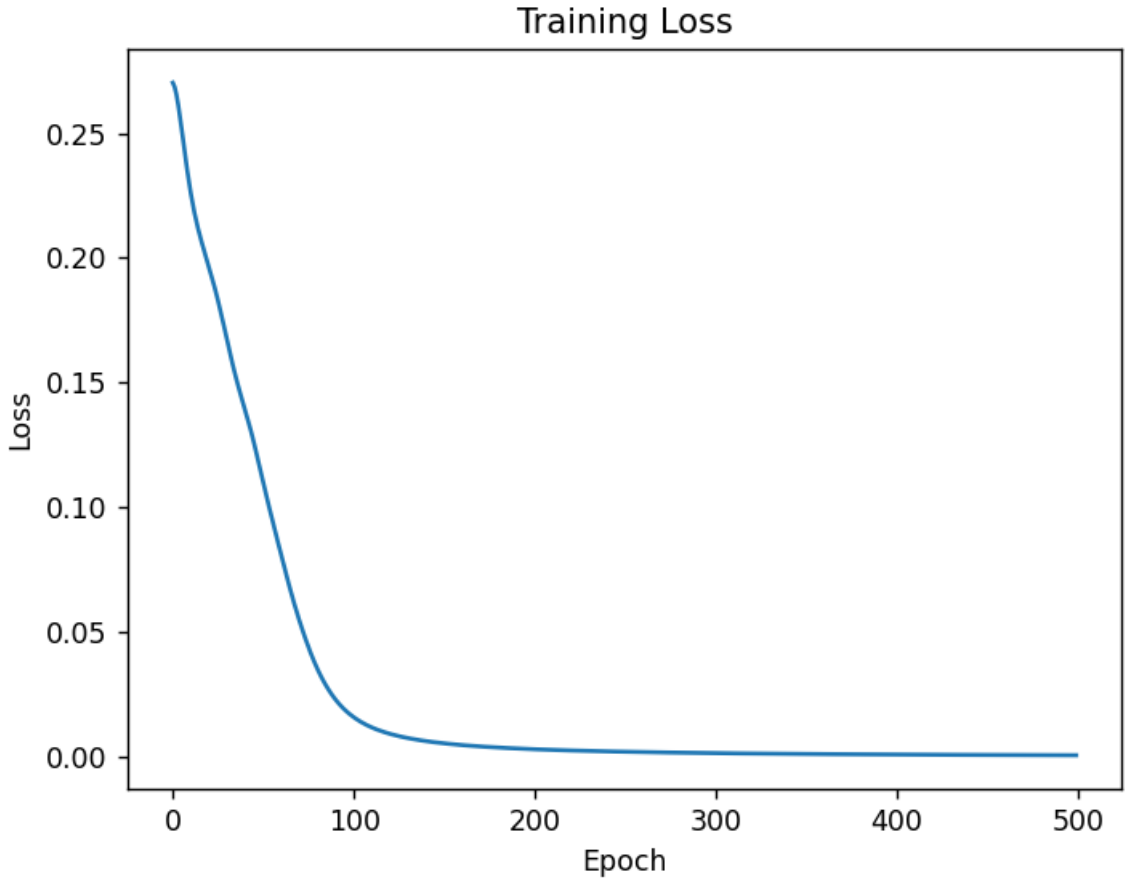



```
Predictions for a trained network:
[[0.23404675]
 [0.65165151]
 [0.66605936]
 [0.46927604]]
```

Pri trénovaní sme dostali nesprávne hodnoty (perfektné výsledky by boli 0, 1, 1, 0)

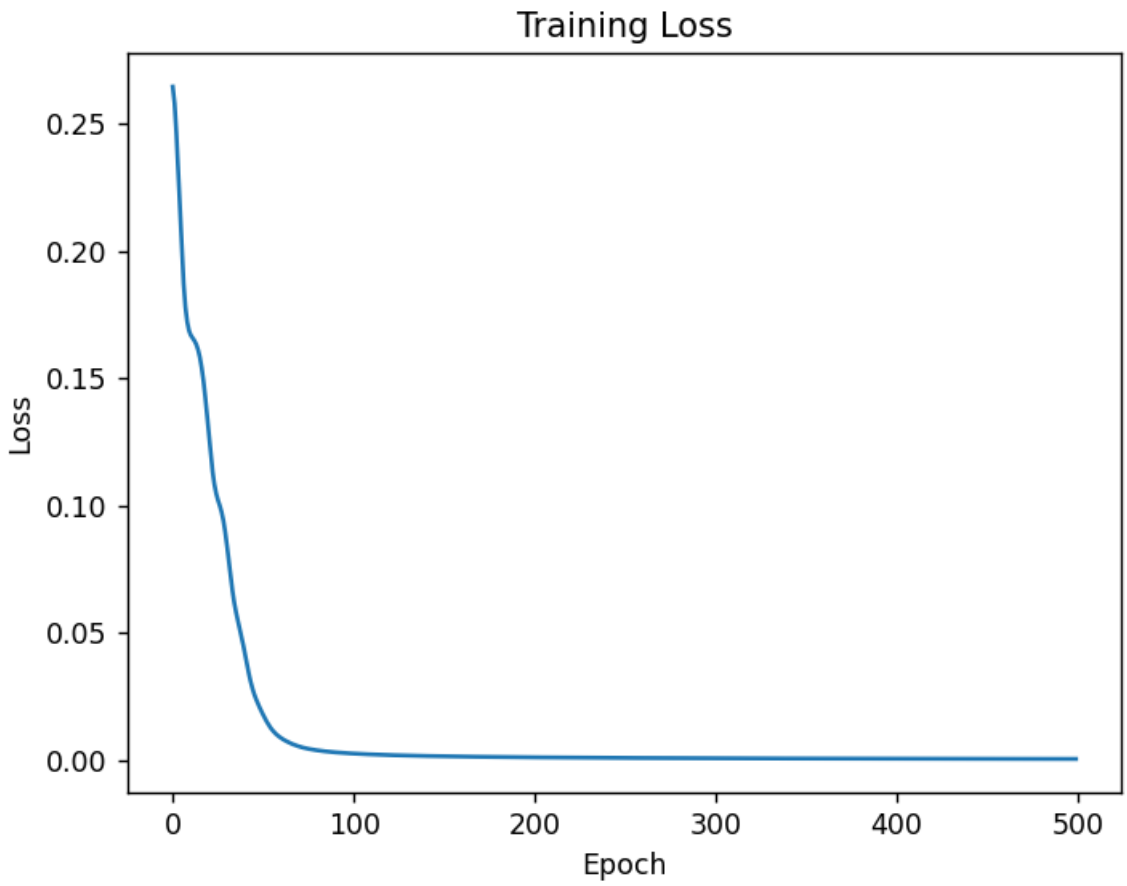
Experiment 5:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
AND	0.9	4	0.8	Tanh



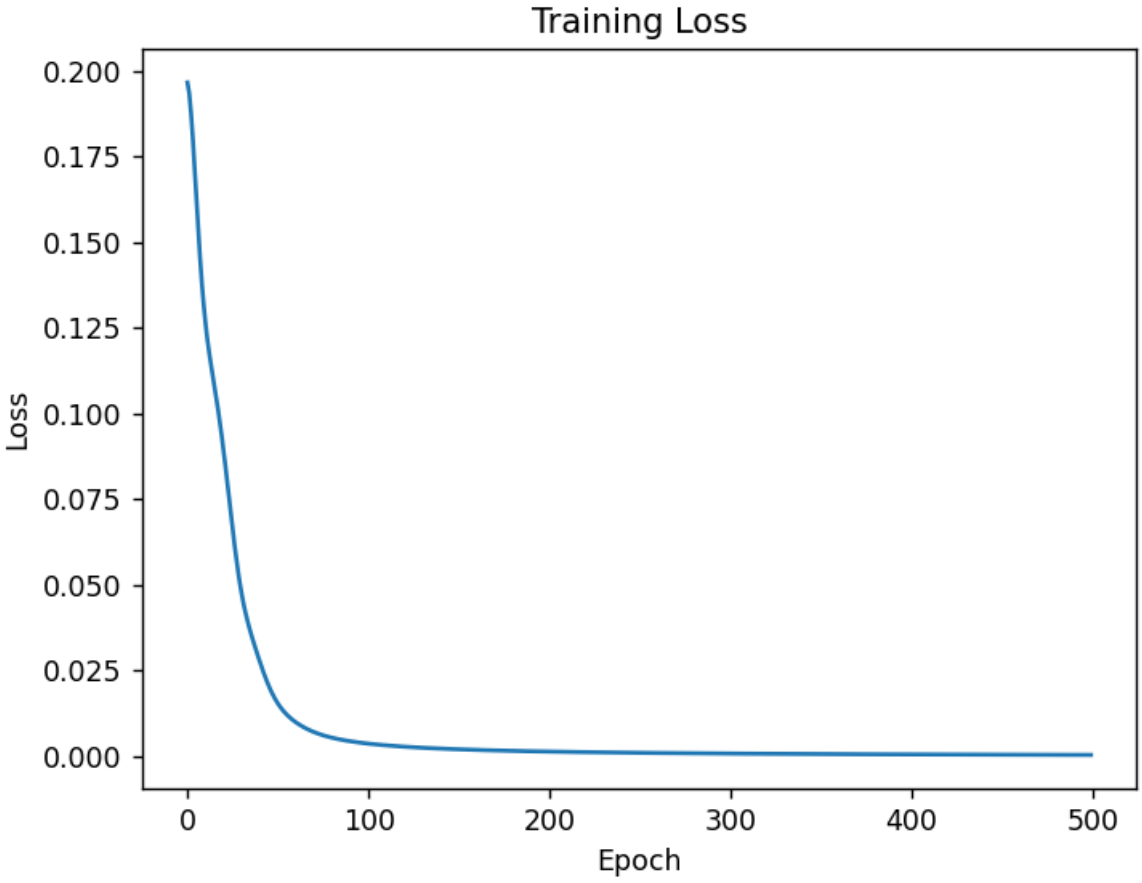
Experiment 6:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
AND	0.9	4, 4	0.8	Tanh



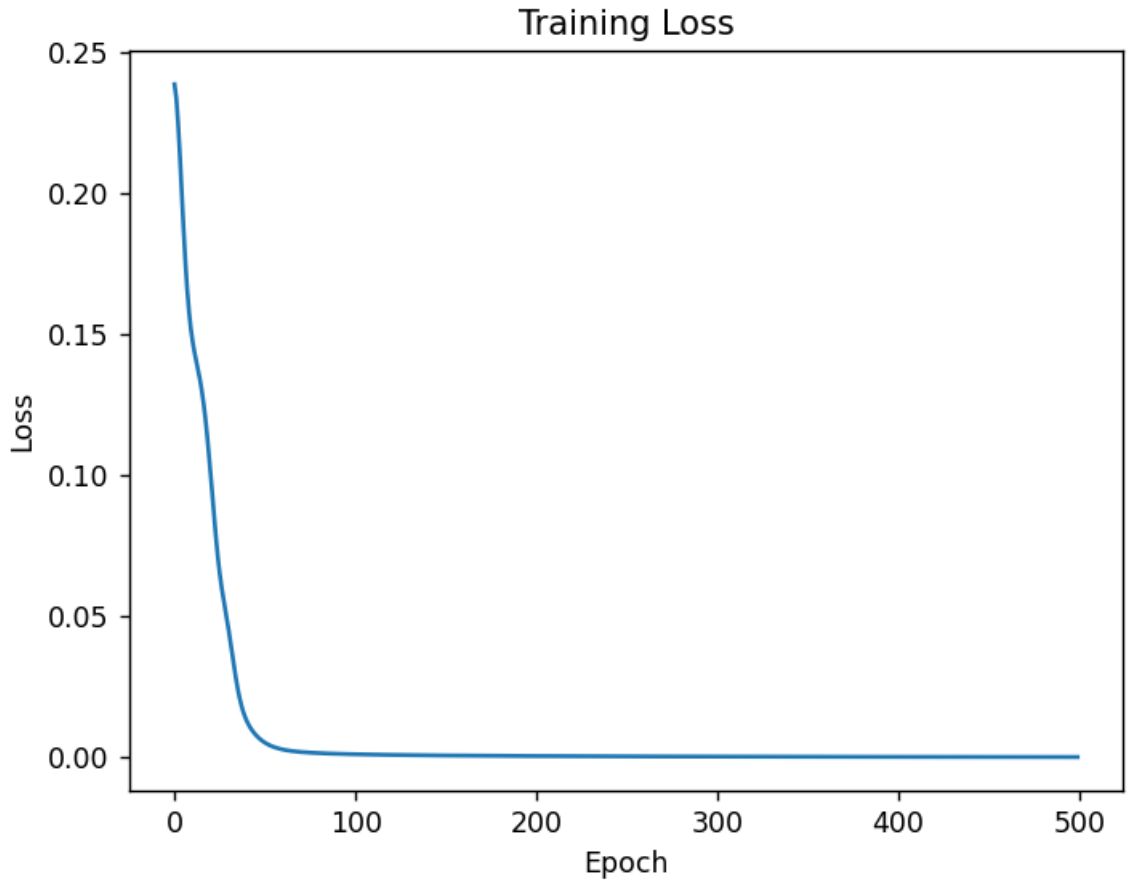
Experiment 7:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
OR	0.9	4	0.8	Tanh



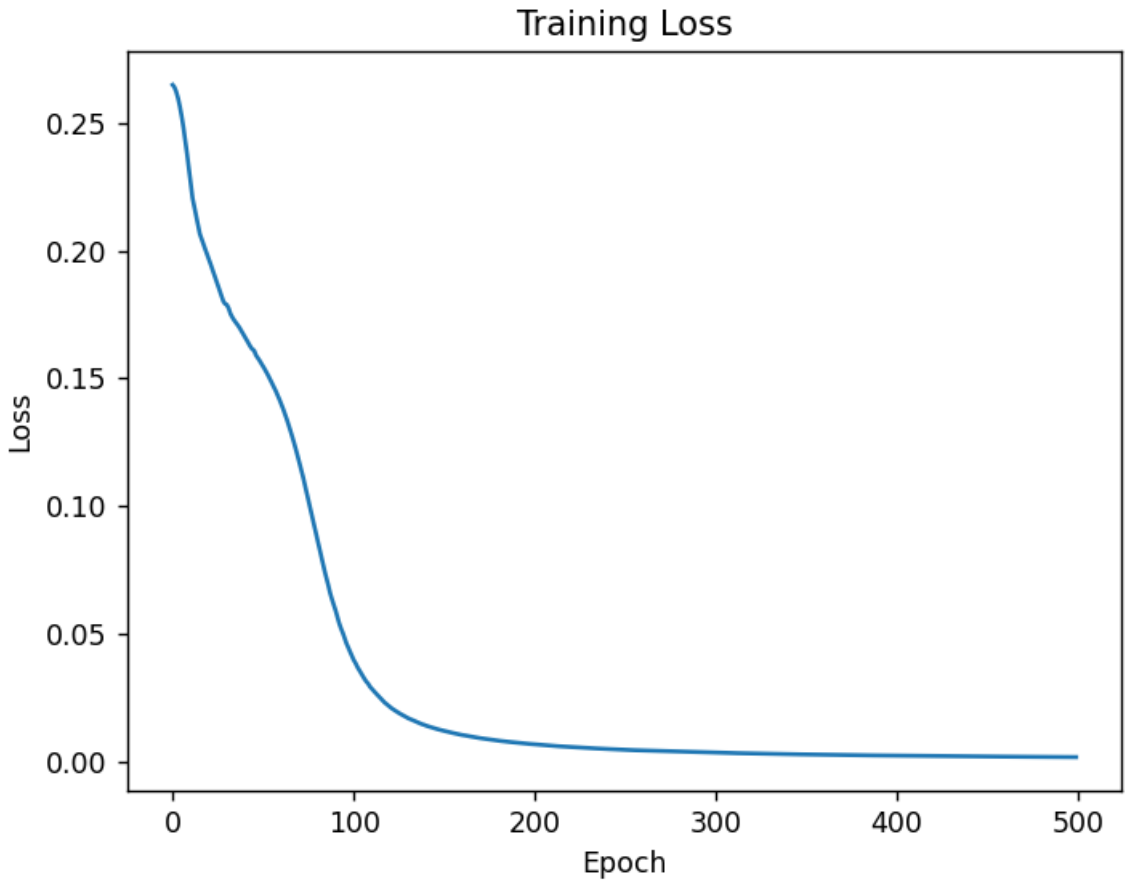
Experiment 8:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
OR	0.9	4, 4	0.8	Tanh



Experiment 9:

Operácia	Moment	Veľkosti skrytých vrstiev	Learning rate	Aktivačné funkcie skrytých funkcií
XOR	0.9	4	0.8	ReLU



Porovnanie experimentov

Pri experimentovaní s rôznymi hodnotami sme sa dozvedeli, že neurónová sieť funguje správne pre všetky tri binárne operácie aj s jednou, aj s dvoma vrstvami. Vynikajúco aj fungovala pri použití ReLU aktivačnej funkcie namiesto Tanh pre skrytú vrstvu.

Problém však môže nastať pri príliš malom momente, kedy sieť dosahuje dobré výsledky chybovej funkcie, ale testovacie výstupy sú príliš odlišné od pravdy.

Najhoršie výsledky sme dostali s neurónovou sieťou nastavenou s malým learning rate. Na grafe tohto experimentu je vidieť, ako sa klesanie vďaka momentu postupne zrýchľuje, ale kvôli malému momentu je 500 epoch nedostatočné množstvo na efektívne vytrénovanie siete.

Používateľská príručka

Po spustení programu si konzola vypýta druh programu, ktorý chceme, aby neurónová sieť riešila. Ako možnosti sa tu nachádzajú problémy XOR, AND a OR.

Po špecifikovaní problému si konzola vypýta ďalšie špecifikácie ohľadom architektúry samotnej neurónovej siete. Nastaviť sa pomocou konzole dá počet skrytých vrstiev, počty neurónov na každej z nich, aktivačné funkcie pre každú z nich ako aj hyperparametre ako learning rate, moment a počet epoch.

Po zadaní všetkých potrebných parametrov sa spustí trénovanie a následné testovanie neurónovej siete na špecifikovanom probléme. Po testovaní vráti výsledky pre jednotlivé testovacie vzorky a vykreslí graf evolúcie testovacej chyby.

Záver

V úlohách sme sa naučili fungovanie neurónových sietí vyskúšaním teoretických princípov na praktickej implementácií v kóde.

Prvá podúloha nás naučila pracovať s často používanou knižnicou na vytváranie neurónových sietí. Tiež nás naučila všeobecné fungovanie neurónovej siete a rozdiely medzi jednotlivými optimalizačnými funkciami.

V druhej podúlohe sme si vyskúšali vlastnú implementáciu celkového vnútorného fungovania neurónovej siete, čo zahrňuje vlastnú implementáciu backpropagácie. To pomohlo k lepšiemu pochopeniu matematických princípov, na ktorých je celý systém neurónových sietí postavený.

Vďaka týmto úlohám sme hlboko rozvinuli obzory v oblasti umelej inteligencie.